

Blind Signatures from Oblivious Transfer

Ky Nguyen¹  and Damien Vergnaud¹

Sorbonne Université, CNRS, LIP6, Paris, France.

Abstract.

1 Preliminaries

Notations. We denote the security parameter by λ . A polynomial time (PT) algorithm $\mathcal{A}$ runs in time polynomial in the (implicit) security parameter λ . We denote “probabilistic polynomial time” by PPT. We write $\text{Time}(\mathcal{A})$ for the runtime of $\mathcal{A}$. A function $f(\lambda)$ is *negligible* in λ if it is $\mathcal{O}(\lambda^{-c})$ for every $c \in \mathbb{N}$. We write $f = \text{negl}(\lambda)$ for short. Similarly, we write $f = \text{poly}(\lambda)$ if $f(\lambda)$ is a polynomial with variable λ . If D is a probability distribution, $x \leftarrow D$ means that x is sampled from D and if S is a set, $x \leftarrow S$ means that x is sampled uniformly and independently at random from S . We also write $|S|$ for the cardinality of set S . Further, we write $D_0 \stackrel{c}{\approx} D_1$ for distributions D_0, D_1 , if for all PPT adversaries $\mathcal{A}$, we have $|\Pr[x_0 \leftarrow D_0 : \mathcal{A}(1^\lambda, x_0) = 1] - \Pr[x_1 \leftarrow D_1 : \mathcal{A}(1^\lambda, x_1) = 1]| = \text{negl}(\lambda)$. Similarly, we write $D_0 \stackrel{s}{\approx} D_1$ if the above holds even for unbounded adversaries. For some PPT algorithm $\mathcal{A}$, we write $\mathcal{A}^\mathcal{O}$ if $\mathcal{A}$ has oracle access to the oracle $\mathcal{O}$. If $\mathcal{A}$ performs some check, and the check fails, we assume that $\mathcal{A}$ outputs $\perp$ immediately. Generally, we assume that adversaries are implicitly stateful. We denote with $[n]$ the set $\{1, \dots, n\}$ for $n \in \mathbb{N}$. We write $\mathbb{P}$ for the set of primes and $\mathbb{P}_I$ for the set of primes in the interval I . For some odd prime p , we use the representatives $\{-\frac{p-1}{2}, \dots, \frac{p-1}{2}\}$ for $\mathbb{Z}_p$. For a group $\mathbb{G}$ we write $\text{ord}(\mathbb{G})$ to denote the order of $\mathbb{G}$ and unless stated otherwise we write $\mathbb{G}$ with additive notation. For a group element g we write $\text{ord}(g)$ to denote the order of the group element. We denote by $\text{QR}_N = \{a \in \mathbb{Z}_N^* : \exists b \in \mathbb{Z}_N^*, b^2 \equiv a \pmod{N}\}$ the quadratic residues $\pmod{N}$. For some $N \in \mathbb{N}$, the group QR_N is a cyclic subgroup of $\mathbb{Z}_N^*$ and we denote by $\text{Gen}(\text{QR}_N)$ the set of generators of QR_N . Some properties of QR_N are recalled in lemma 1 in Section 2.

Probability. Let $V, L \in \mathbb{N}$. We define *uniform rejection sampling* for the interval $[V, (V+1)L]$ with masking overhead L as in [5]. Let $v \in [0, V]$. To mask v additively with a mask μ via rejection sampling, perform the following steps.

1. Draw a random mask $\mu \leftarrow [0, (V+1)L]$.
2. Abort if $v + \mu \notin [V, (V+1)L]$.
3. Output $w = v + \mu$.

The value w is uniform over $[V, (V+1)L]$ conditioned on no abort and the abort probability is at most $1/L$ ¹. We use a version of the Forking Lemma from [1, Lemma 1] that fits our usage of it. The lemma was first introduced by Pointcheval and Stern [11] then generalized in [4, 1]. The formal statement can be found in Section 2.2.

Hardness Assumptions. We use the following assumptions in this paper. Let GenG be a PPT algorithm that on input 1^λ and prime order p , outputs (a description of) a group $\mathbb{G} \leftarrow \text{GenG}(1^\lambda)$ of order p . We generally use additive notations for prime order groups and capital letters for elements. Also, we assume that given the description, group operations and membership tests are efficient. We write $g \leftarrow \mathbb{G}$ for drawing elements from some group $\mathbb{G}$ at random. In the following, we assume that prime order groups are setup with GenG implicitly.

¹ When $v = 0$, the number of “bad” $\mu \in [0, (V+1)L]$ causing abort is $\#\{[0, V-1]\} = V$. For a fixed $0 < v \in (0, V]$, the number of “bad” μ causing abort is $\#\{[0, V-1-v]\} + \#\{[(V+1)L-v+1, (V+1)L]\} = V$. In both cases the abort probability over choices of μ is $V/((V+1)L+1) \leq 1/L$.

Let GenRSA be a PPT algorithm that on input 1^λ outputs $(N, P, Q) \leftarrow \text{GenRSA}(1^\lambda)$ such that $N = P \cdot Q$ with $P, Q \in \mathbb{P}$, where $P = 2P' + 1$ and $Q = 2Q' + 1$ are strong primes (*i.e.*, P', Q' are also primes). We assume that $P', Q' > 2^{\lambda+1}$.

First of all, the (D, ℓ) -relaxed DLOG assumption with regards to $\vec{g}$, where $\vec{g} = (g_0, \dots, g_\ell) \in \text{QR}_N^{\ell+1}$, assumes that for any PPT adversary, given $(N, g_0, \dots, g_\ell)$ it is only with negligible probability to output $(c, d, x_0, \dots, x_\ell)$ satisfying $c^d = \prod_{i=0}^\ell g_i^{x_i} \wedge \exists i: \frac{x_i}{d} \notin \mathbb{Z} \wedge d \in [0, D] \wedge x_i \in \mathbb{Z}$. The (D, ℓ) -relaxed DLOG assumption holds under the strong RSA assumption for all $D \leq 2^{\lambda+1}$. Next, the *Decisional Diffie-Hellman* (DDH) assumption in a cyclic group $\mathbb{G}$ assumes that for all PPT adversary it is only with negligible probability that the adversary can distinguish (aG, bG, abG) from (aG, bG, cG) where $G \leftarrow \mathbb{G}$ and $a, b, c \leftarrow \text{ord}(\mathbb{G})$. Finally, the *strong RSA* (sRSA) assumes that it is only with negligible probability for any PPT adversary to output (e, z) such that $z^e \equiv y \pmod{N}$.

Explaining Random Group Elements as Random Strings. For our framework, we require commitments with uniform public parameters pp . For readability, we allow pp (and also uniform random strings urs of NIZKs) to contain (uniform) group elements g of prime-order groups $\mathbb{G}$ with known order p . This is without loss of generality because with *explainable sampling*, we can explain $g \leftarrow \mathbb{G}$ as a random bitstring. We refer to, *e.g.*, [10, Appendix B] for more details.

1.1 Cryptographic Primitives

Commitment Scheme. A *commitment scheme* is a tuple of PPT algorithms $\text{C} = (\text{C.Setup}, \text{C.Commit}, \text{C.Verify})$ such that

- $\text{C.Setup}(1^\lambda)$: generates the public parameters pp ,
- $\text{C.Commit}(\text{pp}, m)$: given the public parameters pp , message $m \in \mathcal{C}_{\text{msg}}$, computes a commitment $c \in \mathcal{C}_{\text{com}}$ with opening randomness d , and outputs the pair (c, d) ,
- $\text{Verify}(\text{pp}, c, m, d)$: given the public parameters pp , message $m \in \mathcal{C}_{\text{msg}}$, and opening randomness d , outputs a bit $b \in \{0, 1\}$ which depends on the validity of the opening (m, d) with respect to the commitment c .

Here, $\mathcal{C}_{\text{msg}}$, $\mathcal{C}_{\text{rnd}}$, $\mathcal{C}_{\text{com}}$, are message, randomness, and commitment spaces, respectively. If the public parameters are uniform or explainable (*i.e.*, Setup outputs some $\text{pp} \leftarrow \{0, 1\}^\ell$ for $\ell \in \mathbb{N}$) we omit Setup without loss of generality.

We require the correctness, hiding and binding properties for a commitment scheme. A commitment schemes is *correct*, if honest commitments $(c, d) \leftarrow \text{Commit}(\text{pp}, m; r)$ always verify, *i.e.* it holds that $\text{Verify}(\text{pp}, c, m, d) = 1$ where pp are the public parameters. It is *hiding* if it is hard to decide whether an unopened commitment c commits to message m_0 or m_1 , and it is *binding* if it is hard to open commitments c to distinct messages. We can have *computational*, *statistical*, *perfect* variants for hiding and binding properties. The formal definitions can be found in Section 2.5.

(Bounded) Integer Commitments. We refer to a commitment scheme with message space $[\mathbb{A}, \mathbb{B}] \subseteq \mathbb{N}$ as a (bounded) integer commitment scheme. We often omit the term bounded if the message space is clear by context.

ElGamal commitments. We recall ElGamal (EG) over a group $\mathbb{G}$ of prime order p with message space $\mathbb{Z}_p$ [7]. We use additive notation for prime order groups.

- $\text{EG.GenPP}(1^\lambda)$: set $(G, H) \leftarrow \mathbb{G}$ and output $\text{pp} = (G, H)$.
- $\text{EG.Commit}(\text{pp}, m)$: sample $r \leftarrow \mathbb{Z}_p$ and set $c = (mG + rH, rG)$, and output (c, r) .
- $\text{EG.Verify}(\text{pp}, c, m, r)$: check if $c = (mG + rH, rG)$.

Note that the public parameters are uniform and we can sample them via a random oracle to avoid trusted setup. EG commitments are correct, hiding under DDH and perfectly binding.

Remark 1. If in verification of EG, we check that $m \in [0, M]$ for $M < p$, then m is fixed over the integers and we can interpret the commitment as an integer commitment with message space $[0, M] \subseteq \mathbb{N}$.

Pedersen Commitments in $\mathbf{QR}_N$. We recall Pedersen multi-commitments (MPed) over $\mathbf{QR}_N$ with message space $\mathbb{Z}^\ell$ for some $\ell \in \mathbb{N}$ (cf. [6]).

- $\text{MPed.GenPP}(1^\lambda)$: set $(N, P, Q) \leftarrow \text{GenRSA}(1^\lambda)$ and sample ℓ random generators g_i of $\mathbf{QR}_N$, and output $\text{pp} = (N, h, g_1, \dots, g_\ell)$. Note that with (P, Q) , we can check whether g_i generates $\mathbf{QR}_N$.
- $\text{MPed.Commit}(\text{pp}, \vec{m})$: sample $r \leftarrow [0, N \cdot 2^\lambda]$, set $c \leftarrow h^r \cdot \prod_{i=1}^\ell g_i^{m_i} \bmod N$, and output (c, r) .
- $\text{MPed.Verify}(\text{pp}, c, \vec{m}, r)$: check if $c = \pm h^r \cdot \prod_{i=1}^\ell g_i^{m_i} \bmod N$.

MPed commitments are correct, statistically hiding and binding under the factoring assumption (which is implied by sRSA). Throughout this work, we use MPed commitments in $\mathbf{QR}_N$ to enforce in security proofs that values extracted from NIZKs are integers via lemma 4.

Signature Scheme. A signature scheme is a tuple of PPT algorithms $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$ such that

- $\text{KeyGen}(1^\lambda)$: generates a verification key vk and a signing key sk ,
- $\text{Sign}(\text{sk}, m)$: given a signing key sk and a message $m \in \mathcal{S}_{\text{msg}}$, *deterministically* outputs a signature σ ,
- $\text{Verify}(\text{vk}, m, \sigma)$: given a verification key pk and a signature σ on message m , *deterministically* outputs a bit $b \in \{0, 1\}$.

Here, $\mathcal{S}_{\text{msg}}$ is the message space. We define the standard notion of *correctness* and *euf-cma* security. Correctness requires that any honestly generated signature $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ verifies, *i.e.* $\text{Verify}(\text{vk}, m, \sigma) = 1$. The *euf-cma* security imposes that even with oracle accesses to $\text{Sign}(\text{sk}, \cdot)$, no PPT adversary will be able to forge a valid signature σ on a message m that is not queried to $\text{Sign}(\text{sk}, \cdot)$.

Blind Signature Scheme. A (two-move) blind signature scheme is a tuple of PPT algorithms $\text{BS} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ such that

Definition 1 (Blind Signature). A (two-move) blind signature scheme is a tuple of PPT algorithms $\text{PBS} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ such that

- $\text{KG}(1^\lambda)$: generates the verification key bvk and signing key bsk ,
- Sign is split into the following algorithms:
 - $\text{User}(\text{bvk}, m)$: given verification key bvk and message $m \in \mathcal{BS}_{\text{msg}}$, outputs a first message ρ_1 and a state st ,
 - $\text{Signer}(\text{bsk}, \rho_1)$: given signing key bsk and first message ρ_1 , outputs a second message ρ_2 ,
 - $\text{Derive}(\text{st}, \rho_2)$: given state st and second message ρ_2 , outputs a signature σ
- $\text{Verify}(\text{bvk}, m, \sigma)$: given verification key bvk and signature σ on message $m \in \mathcal{BS}_{\text{msg}}$, outputs a bit $b \in \{0, 1\}$.

Here, $\mathcal{BS}_{\text{msg}}$ is the message spaces.

We consider the standard security notions for blind signatures [9]. Below, we define correctness, blindness under *malicious keys*, and one-more unforgeability of a blind signature scheme. Moreover, we will omit the state for better readability on occasion.

Definition 2 (Correctness). A blind signature scheme is correct, if for all messages $m \in \mathcal{BS}_{\text{msg}}$, $(\text{bvk}, \text{bsk}) \leftarrow \text{KG}(1^\lambda)$, $(\rho_1, \text{st}) \leftarrow \text{User}(\text{bvk}, m)$, $\rho_2 \leftarrow \text{Signer}(\text{bsk}, \rho_1)$, $\sigma \leftarrow \text{Derive}(\text{st}, \rho_2)$, it holds that $\text{Verify}(\text{bvk}, m, \sigma) = 1$.

Definition 3 (Blindness Under Malicious Keys). A blind signature scheme is blind under malicious keys if for any PPT adversary $\mathcal{A}$, we have

$$\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = \Pr \left[\begin{array}{l} (\text{bvk}, m_0, m_1) \leftarrow \mathcal{A}(1^\lambda), \text{coin} \leftarrow \{0, 1\}, \\ (\rho_{1,b}, \text{st}_b) \leftarrow \text{User}(\text{bvk}, m_b) \text{ for } b \in \{0, 1\}, \\ (\rho_{2,\text{coin}}, \rho_{2,1-\text{coin}}) \leftarrow \mathcal{A}(\rho_{1,\text{coin}}, \rho_{1,1-\text{coin}}), : \text{coin} = \mathcal{A}(\sigma_0, \sigma_1) \\ \sigma_b \leftarrow \text{Derive}(\text{st}_b, \rho_{2,b}) \text{ for } b \in \{0, 1\}, \\ \text{if } \exists b \text{ s.t. } \text{Verify}(\text{bvk}, m_b, \sigma_b) = 0: \\ \quad \text{then } \sigma_0 = \sigma_1 = \perp, \end{array} \right] - \frac{1}{2} = \text{negl}(\lambda).$$

Definition 4 (One-more Unforgeability). A blind signature scheme is one-more unforgeable if for any $Q = \text{poly}(\lambda)$ and PPT adversary $\mathcal{A}$ that makes at most Q signing queries, we have

$$\text{Adv}_{\mathcal{A}}^{\text{omuf}}(\lambda) = \Pr \left[\begin{array}{l} (\text{bvk}, \text{bsk}) \leftarrow \text{KG}(1^\lambda) \\ \{(m_i, \sigma_i)\}_{i \in [Q+1]} \leftarrow \mathcal{A}^{\text{Signer}(\text{bsk}, \cdot)}(\text{bvk}) : \\ \forall i \neq j \in [Q+1] : \\ m_i \neq m_j \\ \wedge \\ \text{Verify}(\text{bvk}, m_i, \sigma_i) = 1 \end{array} \right] = \text{negl}(\lambda).$$

Σ -Protocol. Let R be an NP relation with statements x and witnesses w . We denote by $\mathcal{L}_R = \{x \mid \exists w \text{ s.t. } (x, w) \in R\}$ the language induced by R . A Σ -protocol for an NP relation R for language $\mathcal{L}_R$ with challenge space $\mathcal{CH}$ is a tuple of PPT algorithms $\Sigma = (\text{Init}, \text{Chall}, \text{Resp}, \text{Verify})$ such that

- $\text{Init}(x, w)$: given a statement $x \in \mathcal{L}_R$, and a witness w such that $(x, w) \in R$, outputs a first flow message (*i.e.*, commitment) Ω and a state st , where we assume st includes x, w ,
- $\text{Chall}()$: samples a challenge $\gamma \leftarrow \mathcal{CH}$ (without taking any input),
- $\text{Resp}(\text{st}, \gamma)$: given a state st and a challenge $\gamma \in \mathcal{CH}$, outputs a third flow message (*i.e.*, response) τ ,
- $\text{Verify}(x, \Omega, \gamma, \tau)$: given a statement $x \in \mathcal{L}_R$, a commitment Ω , a challenge $\gamma \in \mathcal{CH}$, and a response τ , outputs a bit $b \in \{0, 1\}$.

We recall the standard notions of *correctness*, *high-min entropy*, *(non-abort) honest-verifier zero-knowledge*, and *k-special soundness*. A Σ -protocol is correct, if for all $(x, w) \in R$, if for any honestly generated transcripts (Ω, γ, τ) , the verifier accepts, *i.e.* $\text{Verify}(x, \Omega, \gamma, \tau) = 1$. It has high min-entropy if for all $(x, w) \in R$, it is *statistically* hard to predict a honestly generated first flow Ω . It is honest-verifier zero-knowledge (HVZK), if there exists a PPT zero-knowledge simulator Sim such that the distributions of $\text{Sim}(x, \gamma)$ and a honestly generated *non-aborting* transcript with Init initialized with (x, w) are statistically indistinguishable for any $x \in \mathcal{L}_R$, and $\gamma \in \mathcal{CH}$, where the honest execution is conditioned on γ being used as the challenge. Finally, it is *k-special sound*, if there exists a *deterministic* PT extractor Ext such that given k valid transcripts $\{(\Omega, \gamma_i, \tau_i)\}_{i \in [k]}$ for statement x with pairwise distinct challenges $(\gamma_i)_i$, outputs a witness w such that $(x, w) \in R$.

Non-Interactive Zero Knowledge. All formal definitions of the following can be found in Section 2.8. Let $\mathcal{URS} = \{0, 1\}^\ell$ be a set of *uniform random strings* for some $\ell \in \mathbb{N}$ and $\mathcal{SRS}$ be some set of *structured random strings* with efficient membership test ². A NIZK for a relation R with common reference string space $\mathcal{CRS} = \mathcal{SRS} \times \mathcal{URS}$ is a tuple of PPT algorithms $(\text{GenSRS}, \text{Prove}^H, \text{Verify}^H)$, where the latter two are oracle-calling, such that:

- $\text{GenSRS}(1^\lambda)$: outputs a structured reference string $\text{srs} \in \mathcal{SRS}$,
- $\text{Prove}^H(\text{crs}, x, w)$: receives a $\text{crs} = (\text{srs}, \text{urs}) \in \mathcal{CRS}$, a statement x and a witness w , and outputs a proof π ,
- $\text{Verify}^H(\text{crs}, x, \pi)$: receives a $\text{crs} = (\text{srs}, \text{urs}) \in \mathcal{CRS}$, a statement x and a proof π , and outputs a bit $b \in \{0, 1\}$.

We recall that $\mathcal{L}_R = \{x \mid \exists w : (x, w) \in R\}$ denotes the language induced by R . If there is no crs needed, *i.e.* $\mathcal{CRS} = \emptyset$, we then omit crs as an input to Prove and Verify . A NIZK is *correct* if for any $\text{crs} = (\text{srs}, \text{urs})$ with $\text{srs} \leftarrow \text{GenSRS}(1^\lambda)$ and $\text{urs} \leftarrow \mathcal{URS}$, $(x, w) \in R$, and $\pi \leftarrow \text{Prove}^H(\text{crs}, x, w)$, it holds that $\text{Verify}^H(\text{crs}, x, \pi) = 1$. It is *zero-knowledge* if there exists a PPT simulator $\text{Sim} = (\text{Sim}_{\text{crs}}, \text{Sim}_H, \text{Sim}_\pi)$ such that the distributions of $\pi' \leftarrow \text{Sim}_\pi(\text{crs}, x)$ and $\pi \leftarrow \text{Prove}^H(\text{crs}, x, w)$ are computationally indistinguishable for any $(x, w) \in R$. Note that the sub-algorithms of Sim share state. For simulated proofs, the algorithm Sim_H simulates the random oracle and Sim_{crs} simulates the $\text{crs} = (\text{srs}, \text{urs})$, where there is a structured part srs . We also define a notion of *subversion zero-knowledge*, inspired by the notion introduced in [3]. To recall, the second part of the $\text{crs} = (\text{srs}, \text{urs})$ is a random reference string which can later be sampled via a random oracle, and the first part is a structured string srs . For *subversion zero-knowledge*, there is no Sim_{crs}

² This membership test is required for our definition of subversion zero-knowledge. Let H be a random oracle. Note that in general it is difficult to check that some srs was generated via GenSRS . (We allow that $\mathcal{SRS}$ is not equal to the output space of GenSRS .)

anymore and the structured srs can be chosen by $\mathcal{A}$, while urs is sampled uniformly at random by H for the real proofs $\pi \leftarrow \text{Prove}^{\mathsf{H}}(\text{crs}, x, w)$ or by Sim_{H} for the simulated proofs $\pi' \leftarrow \text{Sim}_{\pi}(\text{crs}, x)$. Here we also require that the subverted srs belongs to $\mathcal{SRS}$.

We define *adaptive knowledge soundness*. An NIZK is adaptively knowledge sound for relation³ $\tilde{\mathsf{R}}$ if there exist positive polynomials $p_{\mathsf{T}}, p_{\mathsf{P}}$, constant c , a PPT extractor Ext and a PPT simulator SimCRS so that for any $(\overline{\text{crs}}, \text{td}) \leftarrow \text{SimCRS}(1^\lambda)$, given oracle access to any PPT $\mathcal{A}$ (with explicit random tape ρ and making $Q_{\mathsf{H}} = \text{poly}(\lambda)$ RO queries) that cannot distinguish $\overline{\text{crs}} \in \mathcal{CRS}$ from a real $\text{crs} := (\text{srs} \leftarrow \text{GenSRS}(1^\lambda), \text{urs} \leftarrow \mathcal{URS})$, given $(x, \pi) \leftarrow \mathcal{A}^{\mathsf{H}}(\overline{\text{crs}}; \rho)$, with probability at least $\frac{\mu(\lambda)^c - \text{negl}(\lambda)}{p_{\mathsf{P}}(\lambda, Q_{\mathsf{H}})}$ the extractor finds $w \leftarrow \text{Ext}(\overline{\text{crs}}, \text{td}, x, \pi, \rho, \vec{h})$ where $(x, w) \in \tilde{\mathsf{R}}$. Here, $\vec{h}$ contains the outputs of H , the probability is over the random tape ρ of $\mathcal{A}$, the random tape of SimCRS , and the random choices of H . Also, we require that the runtime of Ext is bounded by $p_{\mathsf{T}}(\lambda, Q_{\mathsf{H}}) \cdot \text{Time}(\mathcal{A})$.

We further define *partial online-extractability* for NIZKs over a relation with statements $x = (x_0, x_1)$ and witnesses $w = (w_0, w_1)$. A NIZK is partially online-extractable if there exists an algorithm $\text{Ext} = (\text{Ext}_1, \text{Ext}_2)$ such that Ext_1 samples a partial statement x_0 uniformly at random along with a trapdoor td and for any PPT adversary that outputs pairs of partial statements $x_{1,i}$ and proofs π_i such that all $((x_0, x_{1,i}), \pi_i)$ verify with probability $\mu(\lambda)$, the extraction algorithm Ext_1 can use the trapdoor to extract partial witnesses $w_{1,i}$ for all statements such that there exist partial witnesses $w_{0,i}$ with probability $\frac{\mu(\lambda) - \text{negl}(\lambda)}{p_{\mathsf{P}}(\lambda, Q_{\mathsf{H}})}$ where p_{P} is a polynomial and Q_{H} is the number of hash queries made by the adversary. Looking forward, we will set the first partial statement x_0 to be the public parameters of a commitment scheme and the extracted witness to be the committed values - where the non-extracted witness is the opening of the commitments.

We also define *(statistical) adaptive subversion soundness*. Note that this notion does not require an extractor for the witness and the srs can be maliciously set up by an adversary, which differs from the standard notion of adaptive soundness. An NIZK is *(statistically) adaptively subversion sound* for relation $\tilde{\mathsf{R}}$ inducing a language $\mathcal{L}_{\tilde{\mathsf{R}}}$ if no (possibly unbounded) adversary, given a urs and access to the RO H , can output a subverted srs , an instance x , and a proof π such that $\text{Verify}^{\mathsf{H}}(\text{crs} := (\text{srs}, \text{urs}), x, \pi) = 1$ but $x \notin \mathcal{L}_{\tilde{\mathsf{R}}}$.

Fiat-Shamir transformation. We recall the Fiat-Shamir transformation [8, 2] to turn a Σ -protocol $\Sigma = (\text{Init}, \text{Chall}, \text{Resp}, \text{Verify})$ that satisfies *correctness*, *high-min entropy*, *honest verifier zero-knowledge*, and *k-special soundness*, into a NIZK $\text{FS}[\Sigma] = (\text{GenSRS}, \text{Prove}^{\mathsf{H}}, \text{Verify}^{\mathsf{H}})$ using a random oracle H that maps to the challenge space $\mathcal{CH}$ of Σ :

- $\text{GenSRS}(1^\lambda)$: outputs the empty string ϵ as we do not require a common reference string and omit crs as an input for other below algorithms,
- $\text{Prove}^{\mathsf{H}}(x, w)$: receives a statement x and a witness w , runs $(\Omega, \text{st}) \leftarrow \text{Init}(x, w)$, computes the challenge $\gamma \leftarrow \mathsf{H}(x, \Omega)$, then computes $\tau \leftarrow \text{Resp}(\text{st}, \gamma)$ and outputs $\pi = (\Omega, \gamma, \tau)$.
- $\text{Verify}^{\mathsf{H}}(x, \pi)$: receives a statement x and a proof $\pi = (\Omega, \gamma, \tau)$, checks that and outputs $b \leftarrow \text{Verify}(x, \Omega, \gamma, \tau) \wedge \gamma = \mathsf{H}(x, \Omega)$.

The resulted NIZK satisfies *correctness*, *adaptive knowledge soundness* and *zero-knowledge*.

2 Full Preliminaries

2.1 Notation

Let $\lambda \in \mathbb{N}$ be the security parameter. A probabilistic polynomial time (PPT) algorithm $\mathcal{A}$ runs in time polynomial in the (implicit) security parameter λ . We write $\text{Time}(\mathcal{A})$ for the runtime of $\mathcal{A}$. A function $f(\lambda)$ is *negligible* in λ if it is $\mathcal{O}(\lambda^{-c})$ for every $c \in \mathbb{N}$. We write $f = \text{negl}(\lambda)$ for short. Similarly, we write $f = \text{poly}(\lambda)$ if $f(\lambda)$ is a polynomial with variable λ . If D is a probability distribution, $x \leftarrow D$ means that x is sampled from D and if S is a set, $x \leftarrow S$ means that x is sampled uniformly and independently at random from S . We also write $|S|$ for the cardinality of set S . Further, we write $D_0 \stackrel{c}{\approx} D_1$ for distributions D_0, D_1 , if for all PPT adversaries $\mathcal{A}$, we

³ We remark that the soundness relation $\tilde{\mathsf{R}}$ can be different from the (correctness) relation R . If $\tilde{\mathsf{R}}$ is not explicitly defined, we implicitly set $\tilde{\mathsf{R}} = \mathsf{R}$.

have $|\Pr[x_0 \leftarrow D_0 : \mathcal{A}(1^\lambda, x_0) = 1] - \Pr[x_1 \leftarrow D_1 : \mathcal{A}(1^\lambda, x_1) = 1]| = \text{negl}(\lambda)$. Similarly, we write $D_0 \stackrel{s}{\approx} D_1$ if the above holds even for unbounded adversaries. For some PPT algorithm $\mathcal{A}$, we write $\mathcal{A}^\mathcal{O}$ if $\mathcal{A}$ has oracle access to the oracle $\mathcal{O}$. If $\mathcal{A}$ performs some check, and the check fails, we assume that $\mathcal{A}$ outputs $\perp$ immediately. Generally, we assume that adversaries are implicitly stateful.

We denote with $[n]$ the set $\{1, \dots, n\}$ for $n \in \mathbb{N}$. We write $\mathbb{P}$ for the set of primes and $\mathbb{P}_I$ for the set of primes in the interval I . For some odd prime p , we use the representatives $\{-\frac{p-1}{2}, \dots, \frac{p-1}{2}\}$ for $\mathbb{Z}_p$. For a group $\mathbb{G}$ we write $\text{ord}(\mathbb{G})$ to denote the order of $\mathbb{G}$ and unless stated otherwise we write $\mathbb{G}$ with additive notation. We denote by $\text{QR}_N = \{a \in \mathbb{Z}_N^* : \exists b \in \mathbb{Z}_N^*, b^2 \equiv a \pmod N\}$ the quadratic residues $\pmod N$. For some $N \in \mathbb{N}$, the group QR_N is a cyclic subgroup of $\mathbb{Z}_N^*$ and we denote by $\text{Gen}(\text{QR}_N)$ the set of generators of QR_N . We recall some properties of QR_N

Lemma 1 (Proposition 1, [6]). *Let $\lambda \in \mathbb{N}$ and $(N, P, Q) \leftarrow \text{GenRSA}(1^\lambda)$. Considering QR_N , the following holds:*

- The group QR_N is cyclic of order $P'Q'$ where $P = 2P' + 1$ and $Q = 2Q' + 1$.
- $-1 \notin \text{QR}_N$.
- Any square $h \in \text{QR}_N$ has exactly four roots, among which there is exactly one square.
- For any element $h \in \text{QR}_N$, finding roots of h is equivalent to factoring N .
- For $g, h \leftarrow \text{QR}_N$, finding $a, b \in \mathbb{N} \setminus \{0\}$ such that $g^a \equiv h^b \pmod N$ is equivalent to factoring N .
- For any $e \in \mathbb{N}$ coprime with $\phi(N)$ and $y \in \mathbb{Z}_N^*$, finding $x, e' \in \mathbb{N}$ such that $x^e \equiv y^{e'} \pmod N$ is equivalent to finding an e -th root of y in $\mathbb{Z}_N^*$.

2.2 Probability

Rejection Sampling. Let $V, L \in \mathbb{N}$. We define uniform rejection sampling for the interval $[0, V]$ with masking overhead L as in [5]. Let $v \in [0, V]$. To mask v additively with a mask μ via rejection sampling, perform the following steps.

1. Draw a random mask $\mu \leftarrow [0, (V+1)L]$.
2. Abort if $v + \mu \notin [V, (V+1)L]$.
3. Output $w = v + \mu$.

The value w is uniform over $[V, (V+1)L]$ conditioned on no abort and the abort probability is at most $1/L$. This is easy to see as it is a requirement for the abort that either

Noise Flooding. Let $V, L \in \mathbb{N}$. We define noise flooding for the interval $[-V, V]$ with masking overhead $L = 2^\lambda$. Let $v \in [-V, V]$. To mask v additively with a mask μ via noise flooding, output $w = v + \mu$, where $\mu \leftarrow [0, VL]$ is sampled at random. The value w is distributed close to uniform over $[0, VL]$ with statistical distance at most $1/L = \text{negl}(\lambda)$.

Forking Lemma. We state here a version of Forking Lemma [11, 4] that fits our usage of it.

Lemma 2 (Lemma 1, [1]). *Let H be a set and let $F : H^q \rightarrow [q]$ be a possibly random function. For every $\vec{h} \in H^q$, let $E(\vec{h})$ be a probability event. The probability that when sampling k vectors $\vec{h}_1, \dots, \vec{h}_k$ uniformly and independently at random (conditioned that vectors are identical on their first $F(\vec{h}_1)$ components), $E(\vec{h}_i)$ happens for all $i \in [k]$ and $F(\vec{h}_1) = F(\vec{h}_2) = \dots = F(\vec{h}_k)$, is at least $\delta(E)^k / q^{k-1}$, where $\delta(E) := \Pr[\vec{h} \leftarrow H^q : E(\vec{h})]$.*

2.3 Assumptions

Groups and RSA. Let GenG be a PPT algorithm that on input 1^λ and prime order p , outputs (a description of) a group $\mathbb{G} \leftarrow \text{GenG}(1^\lambda)$ of order p . We generally use additive notations for prime order groups and capital letters for elements. Also, we assume that given the description, group operations and membership tests are efficient. We write $g \leftarrow \mathbb{G}$ for drawing elements from some group $\mathbb{G}$ at random. In the following, we assume that prime order groups are setup with GenG implicitly.

Let GenRSA be a PPT algorithm that on input 1^λ outputs $(N, P, Q) \leftarrow \text{GenRSA}(1^\lambda)$ such that $N = P \cdot Q$ with $P, Q \in \mathbb{P}$, where $P = 2P' + 1$ and $Q = 2Q' + 1$ are strong primes (i.e., P', Q' are also primes). We assume that $P', Q' > 2^{\lambda+1}$.

Before recalling some standard hardness assumptions, let us recall the following well-known lemma.

Lemma 3. *Given $x, y \in \mathbb{Z}_N^*$ with $a, b \in \mathbb{Z}$ such that $x^a = y^b$ and $\gcd(a, b) = 1$, one can efficiently compute $\bar{x} \in \mathbb{Z}_N^*$ such that $\bar{x}^a = y$.*

Remark 2. We need the following well-known fact. Let $\mathbb{G}$ be a group and let $G \leftarrow \mathbb{G}$ be a random element from $\mathbb{G}$. Let $S \in \mathbb{N}$. We consider the problem of distinguishing zG , where $z \leftarrow [0, S]$, from $\tilde{z}G$ where $\tilde{z} \leftarrow \mathbb{Z}_{\text{ord}(G)}$.

If the order p of the group $\mathbb{G}$ is known, then the distinguishing probability is 0 for $S = p - 1$. If only an upper bound U on the order is known, then the distinguishing probability is upper bounded by $1/L$ for $S = L \cdot U$. For the latter, we set $L = 2^\lambda$ throughout to obtain negligible distinguishing probability.

Next, we recall the definition of a relaxed DLOG-relation from [5] (for the hidden order group QR_N).

Definition 5 (((D, ℓ) -relaxed DLOG-relation). *Let $(N, P, Q) \leftarrow \text{GenRSA}(1^\lambda)$, $D, \ell \in \mathbb{N}$, and $\vec{g} = (g_0, \dots, g_\ell) \in \text{QR}_N^{\ell+1}$. Define the (D, ℓ) -relaxed DLOG relation with regards to $\vec{g}$ as*

$$\text{R}_{D, \ell}(\vec{g}) = \left\{ (c, d, \{x_i\}_{i=1}^\ell) \mid \begin{array}{l} c^d = \prod_{i=0}^\ell g_i^{x_i} \wedge \exists i: \frac{x_i}{d} \notin \mathbb{Z} \\ \wedge d \in [0, D] \wedge x_i \in \mathbb{Z} \end{array} \right\}$$

We define the advantage of $\mathcal{A}$ against the hardness of the (D, ℓ) -relaxed DLOG-relation as

$$\text{Adv}_{(D, \ell), \mathcal{A}}^{\text{rel-dlog}}(\lambda) := \Pr \left[\begin{array}{l} (N, P, Q) \leftarrow \text{GenRSA}(1^\lambda); g_0, \dots, g_\ell \leftarrow \text{Gen}(\text{QR}_N); \\ (c, d, x_0, \dots, x_\ell) \leftarrow \mathcal{A}(N, g_0, \dots, g_\ell): \\ (c, d, x_0, \dots, x_\ell) \in \text{R}_{D, \ell}(\vec{g}) \end{array} \right].$$

The following lemma is a simplification of Lemma A.13 of [5] sufficient for our purpose. Note that $\text{ord}(\text{QR}_N) = P'Q'$ and we assume that $P', Q' > 2^{\lambda+1}$.

Lemma 4. *Let $D \leq 2^{\lambda+1}$ and $\ell = \text{poly}(\lambda)$. For every PPT adversary $\mathcal{A}$ we have that $\text{Adv}_{(D, \ell), \mathcal{A}}^{\text{rel-dlog}}(\lambda) = \text{negl}(\lambda)$ under the strong RSA assumption.*

Definition 6 (Decisional Diffie-Hellman). *In a cyclic group $\mathbb{G}$ of prime order p , which are set up w.r.t a security parameter $\lambda \in \mathbb{N}$, the Decisional Diffie-Hellman (DDH) assumption in $\mathbb{G}$ holds if for all PPT adversary $\mathcal{A}$ the advantage*

$$\begin{aligned} & \left| \Pr[G \leftarrow \mathbb{G}; a, b \leftarrow \mathbb{Z}_p: \mathcal{A}(G, aG, bG, abG) = 1] \right. \\ & \left. - \Pr[G \leftarrow \mathbb{G}; a, b, c \leftarrow \mathbb{Z}_p: \mathcal{A}(G, aG, bG, cG) = 1] \right| \end{aligned}$$

is negligible in λ .

Definition 7 (Strong RSA). *Let $\lambda \in \mathbb{N}$. The strong RSA (sRSA) assumption holds if for all PPT $\mathcal{A}$ the advantage*

$$\text{Adv}_{\mathcal{A}}^{\text{s-rsa}}(\lambda) := \Pr \left[\begin{array}{l} (N, P, Q) \leftarrow \text{GenRSA}(1^\lambda); y \leftarrow \mathbb{Z}_N^* \\ (e, z) \leftarrow \mathcal{A}(N, y): z^e \equiv y \pmod{N} \end{array} \right]$$

is negligible in λ .

2.4 Explaining Random Group Elements as Random Strings

For our framework, we require commitments with uniform public parameters pp . For readability, we allow pp (and also uniform random strings urs of NIZKs) to contain (uniform) group elements g of prime-order groups $\mathbb{G}$ with known order p . This is without loss of generality because with *explainable sampling*, we can explain $g \leftarrow \mathbb{G}$ as a random bitstring.

2.5 Commitment Scheme

Definition 8 (Commitment Scheme). A commitment scheme is a tuple of algorithms $\mathcal{C} = (\mathcal{C}.\text{Commit}, \mathcal{C}.\text{Verify})$ such that

- $\mathcal{C}.\text{Setup}(1^\lambda)$: generates the public parameters pp ,
- $\mathcal{C}.\text{Commit}(\text{pp}, m)$: given the public parameters pp , message $m \in \mathcal{C}_{\text{msg}}$, computes a commitment $c \in \mathcal{C}_{\text{com}}$ with opening randomness d , and outputs the pair (c, d) ,
- $\text{Verify}(\text{pp}, c, m, d)$: given the public parameters pp , message $m \in \mathcal{C}_{\text{msg}}$, and opening randomness d , outputs a bit $b \in \{0, 1\}$ which depends on the validity of the opening (m, d) with respect to the commitment c .

Here, $\mathcal{C}_{\text{msg}}$, $\mathcal{C}_{\text{rnd}}$, $\mathcal{C}_{\text{com}}$, are message, randomness, and commitment spaces, respectively. If the public parameters are uniform or explainable as per Section 2.4 (i.e., Setup outputs some $\text{pp} \leftarrow \{0, 1\}^\ell$ for $\ell \in \mathbb{N}$) we omit Setup without loss of generality.

Below, we define the correctness, hiding and binding properties of a commitment scheme.

Definition 9 (Correctness). A commitment scheme is correct, if for all $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $m \in \mathcal{C}_{\text{msg}}$, $r \in \mathcal{C}_{\text{rnd}}$, $(c, d) \leftarrow \text{Commit}(\text{pp}, m; r)$, it holds that $\text{Verify}(\text{pp}, c, m, d) = 1$.

Definition 10 (Hiding). A commitment scheme is hiding if for any PPT adversary $\mathcal{A}$, we have

$$\text{Adv}_{\mathcal{A}}^{\text{hide}}(\lambda) = \left| \Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda), (m_0, m_1) \leftarrow \mathcal{A}(\text{pp}), \\ m_0, m_1 \in \mathcal{C}_{\text{msg}}, \text{coin} \leftarrow \{0, 1\} \\ (c, d) \leftarrow \text{Commit}(\text{pp}, m_{\text{coin}}), \end{array} : \text{coin} = \mathcal{A}(c) \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

Definition 11 (Binding). A commitment scheme is binding if for any PPT adversary $\mathcal{A}$, we have

$$\text{Adv}_{\mathcal{A}}^{\text{bind}}(\lambda) = \Pr \left[\begin{array}{l} \text{pp} \leftarrow \{0, 1\}^{\ell_c}, \\ (c, m_0, m_1, d_0, d_1) \leftarrow \mathcal{A}(\text{pp}) \end{array} : \begin{array}{l} m_0 \neq m_1 \in \mathcal{C}_{\text{msg}} \\ \text{Verify}(\text{pp}, c, m_b, d_b) = 1, b \in \{0, 1\} \end{array} \right] = \text{negl}(\lambda).$$

Remark 3. A commitment scheme is said to be *perfectly binding* if for any (possibly unbounded) $\mathcal{A}$, it holds that $\text{Adv}_{\mathcal{A}}^{\text{bind}}(\lambda) = 0$.

(Bounded) Integer Commitments. We refer to a commitment scheme with message space $[\mathbb{A}, B] \subseteq \mathbb{N}$ as a (bounded) integer commitment scheme. We often omit the term bounded if the message space is clear by context.

ElGamal commitments. We recall ElGamal (EG) over a group $\mathbb{G}$ of prime order p with message space $\mathbb{Z}_p$ [7]. We use additive notation for prime order groups.

- $\text{EG}.\text{GenPP}(1^\lambda)$: set $(G, H) \leftarrow \mathbb{G} \setminus \{0\}$ and output $\text{pp} = (G, H)$.
- $\text{EG}.\text{Commit}(\text{pp}, m)$: sample $r \leftarrow \mathbb{Z}_p$ and set $c = (mG + rH, rG)$, and output (c, r) .
- $\text{EG}.\text{Verify}(\text{pp}, c, m, r)$: check if $c = (mG + rH, rG)$.

Note that the public parameters are uniform and we can sample them via a random oracle to avoid trusted setup. EG commitments are correct, hiding under DDH and perfectly binding.

Remark 4. If in verification of EG, we check that $m \in [0, M]$ for $M < p$, then m is fixed over the integers and we can interpret the commitment as an integer commitment with message space $[0, M] \subseteq \mathbb{N}$.

Pedersen Commitments in QR_N We recall Pedersen multi-commitments (MPed) over QR_N with message space $\mathbb{Z}^\ell$ for some $\ell \in \mathbb{N}$ [6].

- $\text{MPed}.\text{GenPP}(1^\lambda)$: set $(N, P, Q) \leftarrow \text{Gen}(1^\lambda)$ and sample ℓ random generators g_i of QR_N , and output $\text{pp} = (N, h, g_1, \dots, g_\ell)$. Note that with (P, Q) , we can check whether g_i generates QR_N .
- $\text{MPed}.\text{Commit}(\text{pp}, \vec{m})$: sample $r \leftarrow [0, N \cdot 2^\lambda]$, set $c \leftarrow h^r \cdot \prod_{i=1}^\ell g_i^{m_i} \bmod N$, and output (c, r) .
- $\text{MPed}.\text{Verify}(\text{pp}, c, \vec{m}, r)$: check if $c = \pm h^r \cdot \prod_{i=1}^\ell g_i^{m_i} \bmod N$.

MPed commitments are correct, statistically hiding and binding under the factoring assumption (which is implied by sRSA). Throughout this work, we use MPed commitments in QR_N to enforce in security proofs that values extracted from NIZKs are integers via lemma 4.

2.6 Signature Scheme

Definition 12 (Signature Scheme). A signature scheme is a tuple of PPT algorithms $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$ such that

- $\text{KeyGen}(1^\lambda)$: generates a verification key vk and a signing key sk ,
- $\text{Sign}(\text{sk}, m)$: given a signing key sk and a message $m \in \mathcal{S}_{\text{msg}}$, outputs a signature σ ,
- $\text{Verify}(\text{vk}, m, \sigma)$: given a verification key pk and a signature σ on message m , deterministically outputs a bit $b \in \{0, 1\}$.

Here, $\mathcal{S}_{\text{msg}}$ is the message space.

We define the standard notion of correctness and **euf-cma** security

Definition 13 (Correctness). A signature scheme is correct, if for all $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, $m \in \mathcal{S}_{\text{msg}}$, and $\sigma \leftarrow \text{Sign}(\text{sk}, m)$, it holds that $\text{Verify}(\text{vk}, m, \sigma) = 1$.

Definition 14 (EUF-CMA). A signature scheme is **euf-cma** if for any PPT adversary $\mathcal{A}$, we have

$$\text{Adv}_{\mathcal{A}}^{\text{euf}}(\lambda) = \Pr \left[\begin{array}{l} (\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (m, \sigma) \leftarrow \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(\text{vk}) \end{array} : m \notin L \wedge \text{Verify}(\text{vk}, m, \sigma) = 1 \right] = \text{negl}(\lambda),$$

where L is the list of messages $\mathcal{A}$ queried to the **Sign**-oracle.

2.7 Σ -Protocol

Let R be an NP relation with statements x and witnesses w . We denote by $\mathcal{L}_R = \{x \mid \exists w \text{ s.t. } (x, w) \in R\}$ the language induced by R . A Σ -protocol for an NP relation R for language $\mathcal{L}_R$ is a tuple of PPT algorithms $\Sigma = (\text{Init}, \text{Chall}, \text{Resp}, \text{Verify})$ such that

- $\text{Init}(x, w)$: given a statement $x \in \mathcal{L}_R$, and a witness w such that $(x, w) \in R$, outputs a first flow message (i.e., commitment) Ω and a state st , where we assume st includes x, w ,
- $\text{Chall}()$: samples a challenge $\gamma \leftarrow \mathcal{CH}$ (without taking any input),
- $\text{Resp}(\text{st}, \gamma)$: given a state st and a challenge $\gamma \in \mathcal{CH}$, outputs a third flow message (i.e., response) τ ,
- $\text{Verify}(x, \Omega, \gamma, \tau)$: given a statement $x \in \mathcal{L}_R$, a commitment Ω , a challenge $\gamma \in \mathcal{CH}$, and a response τ , outputs a bit $b \in \{0, 1\}$.

Definition 15 (Correctness). A Σ -protocol is correct, if for all $(x, w) \in R$, $(\Omega, \text{st}) \leftarrow \text{Init}(x, w)$, $\gamma \in \mathcal{CH}$, and $\tau \leftarrow \text{Resp}(\text{st}, \gamma)$, it holds that $\text{Verify}(x, \Omega, \gamma, \tau) = 1$.

Definition 16 (High Min-Entropy). A Σ -protocol has high min-entropy if for all $(x, w) \in R$ and (possibly unbounded) adversary $\mathcal{A}$, it holds that

$$\Pr[(\Omega, \text{st}) \leftarrow \text{Init}(x, w), \Omega' \leftarrow \mathcal{A}(1^\lambda) : \Omega = \Omega'] = \text{negl}(\lambda).$$

Definition 17 (Non-abort HVZK). A Σ -protocol is non-abort honest-verifier zero-knowledge (HVZK), if there exists a PPT zero-knowledge simulator Sim such that the distributions of $\text{Sim}(x, \gamma)$ and the honestly generated transcript with Init initialized with (x, w) are statistically indistinguishable for any $x \in \mathcal{L}_R$, and $\gamma \in \mathcal{CH}$, where the honest execution is conditioned on γ being used as the challenge and no abort occurring.

We write HVZK for short if the Σ -protocol never aborts.

Definition 18 (k -Special Soundness). A Σ -protocol is k -special sound, if there exists a deterministic PT extractor Ext such that given k valid transcripts $\{(\Omega, \gamma_i, \tau_i)\}_{i \in [k]}$ for statement x with pairwise distinct challenges $(\gamma_i)_i$, outputs a witness w such that $(x, w) \in R$.

2.8 Non-Interactive Zero Knowledge

Let $\mathcal{URS} = \{0, 1\}^\ell$ be a set of *uniform random strings* for some $\ell \in \mathbb{N}$ and $\mathcal{SR}\mathcal{S}$ be some set of *structured random strings* with efficient membership test ⁴. An NIZK for a relation R with common reference string space $\mathcal{CR}\mathcal{S} = \mathcal{SR}\mathcal{S} \times \mathcal{URS}$ is a tuple of PPT algorithms $(\text{GenSRS}, \text{Prove}^H, \text{Verify}^H)$, where the latter two are oracle-calling, such that:

- $\text{GenSRS}(1^\lambda)$: outputs a structured reference string $\text{srs} \in \mathcal{SR}\mathcal{S}$,
- $\text{Prove}^H(\text{crs}, x, w)$: receives a $\text{crs} = (\text{srs}, \text{urs}) \in \mathcal{CR}\mathcal{S}$, a statement x and a witness w , and outputs a proof π ,
- $\text{Verify}^H(\text{crs}, x, \pi)$: receives a $\text{crs} = (\text{srs}, \text{urs}) \in \mathcal{CR}\mathcal{S}$, a statement x and a proof π , and outputs a bit $b \in \{0, 1\}$.

We recall that $\mathcal{L}_R = \{x \mid \exists w : (x, w) \in R\}$ denotes the language induced by R . If there is no crs needed, *i.e.* $\mathcal{CR}\mathcal{S} = \emptyset$, we then omit crs as an input to Prove and Verify .

Definition 19 (Correctness). *An NIZK is correct if for any $\text{crs} = (\text{srs}, \text{urs})$ with $\text{srs} \leftarrow \text{GenSRS}(1^\lambda)$ and $\text{urs} \leftarrow \mathcal{URS}$, $(x, w) \in R$, and $\pi \leftarrow \text{Prove}^H(\text{crs}, x, w)$, it holds that $\text{Verify}^H(\text{crs}, x, \pi) = 1$.*

Definition 20 (Zero-Knowledge). *An NIZK is zero-knowledge (ZK) if there exists a PPT simulator $\text{Sim} = (\text{Sim}_{\text{crs}}, \text{Sim}_H, \text{Sim}_\pi)$ such that for any PPT adversary $\mathcal{A}$, it holds that*

$$\text{Adv}_{\mathcal{A}}^{\text{zk}}(\lambda) = \left| \Pr \left[\begin{array}{l} \text{srs} \leftarrow \text{GenSRS}(1^\lambda), \\ \text{crs} = (\text{srs}, \text{urs}), \\ \mathcal{A}^{H, \mathcal{P}}(\text{crs}) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Sim}_{\text{crs}}(1^\lambda), \\ \text{crs} = (\text{srs}, \text{urs}), \\ \mathcal{A}^{\text{Sim}_H, \mathcal{S}}(\text{crs}) = 1 \end{array} \right] \right| = \text{negl}(\lambda),$$

where $\mathcal{P}$ and $\mathcal{S}$ are oracles that on input (x, w) return $\perp$ if $(x, w) \notin R$, and else output $\text{Prove}^H(\text{crs}, x, w)$ or $\text{Sim}_\pi(\text{crs}, x)$ respectively. Note that the probability is taken over the randomness of Sim and $\mathcal{A}$, and the random choices of H and urs . Also, Sim_{crs} , Sim_H and Sim_π have a shared state.

We also define a notion of *subversion zero-knowledge*, inspired by the notion introduced in [3]. Informally, it guarantees that zero-knowledge holds even for a malicious crs .

Definition 21 (Subversion Zero-Knowledge). *An NIZK is subversion zero-knowledge (Sub-ZK) if there exists a PPT simulator $\text{Sim} = (\text{Sim}_H, \text{Sim}_\pi)$ such that for any PPT adversary $\mathcal{A}$, it holds that*

$$\text{Adv}_{\mathcal{A}}^{\text{sub-zk}}(\lambda) = \left| \Pr \left[\begin{array}{l} \text{urs} \leftarrow \mathcal{URS}, \\ (\text{srs}, \text{st}) \leftarrow \mathcal{A}^H(\text{urs}), \\ \text{crs} = (\text{srs}, \text{urs}), \\ \mathcal{A}^{H, \mathcal{P}}(\text{st}) = 1 \wedge \text{srs} \in \mathcal{SR}\mathcal{S} \end{array} \right] - \Pr \left[\begin{array}{l} \text{urs} \leftarrow \mathcal{URS}, \\ (\text{srs}, \text{st}) \leftarrow \mathcal{A}^{\text{Sim}_H}(\text{urs}), \\ \text{crs} = (\text{srs}, \text{urs}), \\ \mathcal{A}^{\text{Sim}_H, \mathcal{S}}(\text{st}) = 1 \wedge \text{srs} \in \mathcal{SR}\mathcal{S} \end{array} \right] \right| = \text{negl}(\lambda),$$

where $\mathcal{P}$ and $\mathcal{S}$ are oracles that on input (x, w) return $\perp$ if $(x, w) \notin R$, and else output $\text{Prove}^H(\text{crs}, x, w)$ or $\text{Sim}_\pi(\text{crs}, x)$, respectively. Note that the probability is taken over the randomness of Sim and $\mathcal{A}$, and the random choices of H and urs . Also, both Sim_H and Sim_π have a shared state.

We define different notions of soundness. We remark that the soundness relation $\tilde{R}$ can be different from the (correctness) relation R . If $\tilde{R}$ is not explicitly defined, we implicitly set $\tilde{R} = R$.

Definition 22 (Adaptive Knowledge Soundness). *An NIZK is adaptively knowledge sound for relation $\tilde{R}$ if there exists PPT simulator SimCRS and extractor Ext such that*

CRS Indistinguishability. *For any PPT adversary $\mathcal{A}$, we have*

$$\text{Adv}_{\mathcal{A}}^{\text{crs}}(\lambda) = \left| \Pr \left[\begin{array}{l} \text{srs} \leftarrow \text{GenSRS}(1^\lambda), \text{urs} \leftarrow \mathcal{URS}, \\ \text{crs} = (\text{srs}, \text{urs}) : \mathcal{A}^H(\text{crs}) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} (\overline{\text{crs}}, \text{td}) \leftarrow \text{SimCRS}(1^\lambda) : \\ \mathcal{A}^H(\overline{\text{crs}}) = 1 \end{array} \right] \right| = \text{negl}(\lambda),$$

⁴ This membership test is required for our definition of subversion zero-knowledge. Note that in general it is difficult to check that some srs was generated via GenSRS . (We allow that $\mathcal{SR}\mathcal{S}$ is not equal to the output space of GenSRS .)

Knowledge Soundness. *There exists positive polynomials p_T, p_P and a constant c such that given oracle access to any PPT adversary $\mathcal{A}$ (with explicit random tape ρ) that makes $Q_H = \text{poly}(\lambda)$ random oracle queries with*

$$\Pr[(\overline{\text{crs}}, \text{td}) \leftarrow \text{SimCRS}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}^H(\overline{\text{crs}}; \rho) : \text{Verify}^H(\overline{\text{crs}}, x, \pi) = 1] \geq \mu(\lambda),$$

we have

$$\Pr \left[\begin{array}{l} (\overline{\text{crs}}, \text{td}) \leftarrow \text{SimCRS}(1^\lambda), \\ (x, \pi) \leftarrow \mathcal{A}^H(\overline{\text{crs}}; \rho), \\ w \leftarrow \text{Ext}(\overline{\text{crs}}, \text{td}, x, \pi, \rho, \vec{h}) \end{array} : (x, w) \in \tilde{R} \right] \geq \frac{\mu(\lambda)^c - \text{negl}(\lambda)}{p_P(\lambda, Q_H)},$$

where $\vec{h}$ are the outputs of H , and the probability is over the random tape ρ of $\mathcal{A}$, the random tape of SimCRS , and the random choices of H . Also, we require that the runtime of Ext is bounded by $p_T(\lambda, Q_H) \cdot \text{Time}(\mathcal{A})$.

We also adapt the standard notion of online-extractability in two ways. Instead of embedding the online-extraction trapdoor td into crs , we allow that the extractor embeds it into specific parts of statement. Also, we relax the requirements in the sense that only a partial witness w_1 is extracted. For extraction, we require that there exists a witness w_0 such that $(x, (w_0, w_1)) \in \tilde{R}$.

Definition 23 (Partial Online Extractability). *An NIZK is partially online-extractable for relation $\tilde{R}$ with statements $x = (x_0, x_1)$ and witnesses $w = (w_0, w_1)$, where $w_0 \in W_0$ and $x_0 \in X_0$ for some sets W_0, X_0 , if for all PPT adversaries $\mathcal{A}$, there exists a stateful PPT extractor $\text{Ext} = (\text{Ext}_1, \text{Ext}_2)$, such that*

1. x_0 is distributed uniform over X_0 for $(x_0, \text{td}) \leftarrow \text{Ext}_1(1^\lambda)$ and
2. there exists positive polynomials p_T, p_P such that for any $Q_H = \text{poly}(\lambda)$ and PPT adversary $\mathcal{A}$ that makes at most Q_H random oracle queries with

$$\Pr \left[\begin{array}{l} (x_0, \text{td}) \leftarrow \text{Ext}_1(1^\lambda), \text{crs} \leftarrow \text{GenSRS}(1^\lambda), \\ \{(x_{1,i}, \pi_i)\}_{i \in [Q_S]} \leftarrow \mathcal{A}^H(\text{crs}, x_0), x_i \leftarrow (x_0, x_{1,i}) : \\ \forall i \in [Q_S] : \text{Verify}^H(\text{crs}, x_i, \pi_i) = 1 \end{array} \right] \geq \mu(\lambda),$$

it holds that

$$\Pr \left[\begin{array}{l} (x_0, \text{td}) \leftarrow \text{Ext}_1(1^\lambda), \text{crs} \leftarrow \text{GenSRS}(1^\lambda), \\ \{(x_{1,i}, \pi_i)\}_{i \in [Q_S]} \leftarrow \mathcal{A}^H(\text{crs}, x_0), x_i \leftarrow (x_0, x_{1,i}) \\ \{w_{1,i} \leftarrow \text{Ext}_2(\text{crs}, \text{td}, x_i, \pi_i)\}_{i \in [Q_S]} : \\ \forall i \in [Q_S] \exists w_{0,i} \in W_0 : (x_i, (w_{0,i}, w_{1,i})) \in \tilde{R} \\ \wedge \text{Verify}^H(\overline{\text{crs}}, x_i, \pi_i) = 1 \end{array} \right] \geq \frac{\mu(\lambda) - \text{negl}(\lambda)}{p_P(\lambda, Q_H)},$$

where the runtime of Ext is upper bounded by $p_T(\lambda, Q_H) \cdot \text{Time}(\mathcal{A})$.

Adaptive Subversion Soundness. We also define adaptive subversion soundness, where we allow that srs can be maliciously set up by an adversary. Note that this notion does not require an extractor for the witness.

Definition 24 (Statistical Adaptive Subversion Soundness). *An NIZK is (statistically) adaptively sound for relation $\tilde{R}$ inducing a language $\mathcal{L}_{\tilde{R}}$ if for any possibly unbounded $\mathcal{A}$ we have*

$$\text{Adv}_{\mathcal{A}}^{\text{snd}}(\lambda) := \Pr \left[\begin{array}{l} \text{urs} \leftarrow \mathcal{URS}, \\ (\text{srs}, x, \pi) \leftarrow \mathcal{A}^H(1^\lambda; \text{urs}), \\ \text{crs} \leftarrow (\text{srs}, \text{urs}) \end{array} : \begin{array}{l} x \notin \mathcal{L}_{\tilde{R}}, \\ \text{Verify}^H(\text{crs}, x, \pi) = 1 \end{array} \right] \leq \text{negl}(\lambda),$$

where the probability is over the random coins of $\mathcal{A}$ and GenCRS , the random choices of urs , and the random choices of H .

Fiat-Shamir transformation. We recall the Fiat-Shamir transformation [8] to turn a Σ -protocol into a NIZK. Sometimes, we require more involved variants of this transformations. In that case, we provide the compiled NIZK explicitly.

Theorem 1. *Let $\Sigma = (\text{Init}, \text{Chall}, \text{Resp}, \text{Verify})$ be a Σ -protocol that satisfies correctness, high-min entropy, honest verifier zero-knowledge, and 2-Special Soundness. The Fiat-Shamir transformation $FS[\Sigma] = (\text{GenSRS}, \text{Prove}^H, \text{Verify}^H)$ is described below:*

- $\text{GenSRS}(1^\lambda)$: outputs the empty string ϵ as we do not require a common reference string and omit crs as an input for other below algorithms,
- $\text{Prove}^H(x, w)$: receives a statement x and a witness w , runs $(\Omega, \text{st}) \leftarrow \text{Init}(x, w)$, computes the challenge $\gamma \leftarrow H(x, \Omega)$, then computes $\tau \leftarrow \text{Resp}(\text{st}, \gamma)$ and outputs $\pi = (\Omega, \gamma, \tau)$.
- $\text{Verify}^H(x, \pi)$: receives a statement x and a proof $\pi = (\Omega, \gamma, \tau)$, and outputs $b \leftarrow \text{Verify}(x, \Omega, \gamma, \tau) \wedge \gamma = H(x, \Omega)$.

In the ROM, $FS[\Sigma]$ is a NIZK that is correct and satisfies adaptive knowledge soundness.

3 Constructions from OTs

The construction BS_{fis} is given in Figure 1. The public parameters are set to contain:

- A prime-order bilinear group setting $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, g_1, g_2, g_t, \mathbf{e}, p)$ and $\mathbb{G}_1, \mathbb{G}_2$ are both written multiplicatively.
- A random $h_s \in \mathbb{G}_1$, wit independent $\vec{u} := (u_i)_{i=0}^\ell \in \mathbb{G}_1^{\ell+1}$ for the Waters hash function $\mathcal{F}(\mathcal{M}) := u_0 \cdot \prod_i u_i^{M_i}$ on messages $\mathcal{M} := (M_i)_i$.
- A vector $\text{ek} := (h_i)_i := (g_1^{x_i})_i$ and $g_s := \prod_i h_i$, by choosing ℓ random scalars $(x_i)_{i=1}^\ell \in \mathbb{Z}_p^\ell$.

Signer(bvk, bsk)	User(bvk, M)
	1 : $\forall i \in [\ell] : y_i \leftarrow \mathbb{Z}_p$ 2 : $\forall i \in [\ell] : \text{ek}_{i, M_i} := g_1^{y_i}; \text{ek}_{i, 1-M_i} := g_1^{x_i - y_i}$ $(\text{ek}_{i, M_i}, \text{ek}_{i, 1-M_i})_i$ ←
3 : check $\forall i \in [\ell], \text{ek}_{i, 0} \cdot \text{ek}_{i, 1} \stackrel{?}{=} g_1^{x_i}$ 4 : $a \leftarrow \mathbb{Z}_p$, sample α_i s.t. $\prod_i \alpha_i = h_s^a$ 5 : (do ElGamal encryption) $r, t \leftarrow \mathbb{Z}_p$ 6 : $\text{ct}_{i, 0} := \text{ek}_{i, 0}^r \cdot (\alpha_i)^t$ 7 : $\text{ct}_{i, 1} := \text{ek}_{i, 1}^r \cdot (\alpha_i \cdot u_i)^t$, $(\text{ct}_{i, 0}, \text{ct}_{i, 1})_{i \in [\ell]}, \text{ct}_0 := g_1^r, \sigma_2 := (g_2^t)$ →	8 : $\forall i \in [\ell] \sigma_{1, i} := \text{ct}_{i, M_i} / \text{ct}_0^{y_i}$ 9 : (derive Waters signature) $\sigma_1 := \prod_i \sigma_{1, i}$ 10 : (randomize) $\widehat{\sigma}_1 := \sigma_1 \cdot \mathcal{F}(\mathcal{M})^{t'}$; $\widehat{\sigma}_2 := \sigma_2 \cdot g_2^{t'}$; [Ky: NIZK here to prove randomization ?] [Ky: potentially Groth-Sahai stle ?] 11 : return $\sigma = (\mathcal{F}(\mathcal{M}), \widehat{\sigma}_1, \widehat{\sigma}_2)$

Fig. 1: A signing session of BS for message m . We have $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, g_1, g_2, g_t, \mathbf{e}, p, \text{ek}, g_s, h_s, \vec{u})$. The keys are $\text{bvk} = (g_2^a)$, $\text{bsk} = h_s^a$. If a check fails, the party aborts.

Correctness. The derived $\sigma = (\sigma_1, \sigma_2)$ is a Waters signature. Verification using $\text{bvk} = (g_2^a)$ by:

$$\mathbf{e}(h_s, \text{bvk}) \cdot \mathbf{e}(\mathcal{F}(\mathcal{M}), \widehat{\sigma_2}) \stackrel{?}{=} \mathbf{e}(\widehat{\sigma_1}, g_2) .$$

Honest-Verifier Blindness. sttpe:

- ZK simulation for the randomization step, removing dependence on $\mathcal{F}(\mathcal{M}), t'$ w.r.t $\widehat{\sigma_1}, \widehat{\sigma_2}$.
- simulate $\sigma_1 := \widehat{\sigma_1} / \mathcal{F}(\vec{0})^{t_0}$ where $\mathcal{F}(\vec{0})$ and t_0 (comes from the ZK simulator) are independent from $\mathcal{M}$. Output $\mathcal{F}(\vec{0})$ in the derived signature.
- then sample $\sigma_{1,i}$ so that $\prod_i \sigma_{1,i} = \sigma_1$. Now the derivation step does not rely on $\mathcal{M}$ anymore.
- Make the first flow indep from $\mathcal{M}$: sample $\text{ek}_{i,0}, \text{ek}_{i,1}$ such that their product is $g_1^{x_i}$ (part of public parametre.)

References

1. Abe, M., Ambrona, M., Bogdanov, A., Ohkubo, M., Rosen, A.: Acyclicity programming for sigma-protocols. In: Nissim, K., Waters, B. (eds.) TCC 2021, Part I. LNCS, vol. 13042, pp. 435–465. Springer, Heidelberg (Nov 2021). https://doi.org/10.1007/978-3-030-90459-3_15
2. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs. In: Kiltz, E., Vaikuntanathan, V. (eds.) TCC 2022, Part I. LNCS, vol. 13747, pp. 113–142. Springer, Heidelberg (Nov 2022). https://doi.org/10.1007/978-3-031-22318-1_5
3. Bellare, M., Fuchsbaauer, G., Scafuro, A.: NIZKs with an untrusted CRS: Security in the face of parameter subversion. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part II. LNCS, vol. 10032, pp. 777–804. Springer, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53890-6_26
4. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 390–399. ACM Press (Oct / Nov 2006). <https://doi.org/10.1145/1180405.1180453>
5. Couteau, G., Goudarzi, D., Klooß, M., Reichle, M.: Sharp: Short relaxed range proofs. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 609–622. ACM Press (Nov 2022). <https://doi.org/10.1145/3548606.3560628>
6. Couteau, G., Peters, T., Pointcheval, D.: Removing the strong RSA assumption from arguments over the integers. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 321–350. Springer, Heidelberg (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_11
7. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. In: Blakley, G.R., Chaum, D. (eds.) CRYPTO'84. LNCS, vol. 196, pp. 10–18. Springer, Heidelberg (Aug 1984)
8. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO'86. LNCS, vol. 263, pp. 186–194. Springer, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
9. Juels, A., Luby, M., Ostrovsky, R.: Security of blind digital signatures (extended abstract). In: Kaliski Jr., B.S. (ed.) CRYPTO'97. LNCS, vol. 1294, pp. 150–164. Springer, Heidelberg (Aug 1997). <https://doi.org/10.1007/BFb0052233>
10. Katsumata, S., Reichle, M., Sakai, Y.: Practical round-optimal blind signatures in the rom from standard assumptions. to appear in Asiacrypt (2023), <https://eprint.iacr.org/2023/1447>, <https://eprint.iacr.org/2023/1447>
11. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. Journal of Cryptology **13**(3), 361–396 (Jun 2000). <https://doi.org/10.1007/s001450010003>

Supplementary Material